
CS204: Algorithms & Data Structures Lab

Week # 06 (1 Questions, 100 Marks)

Lab session: AL1

Held on: 02-Sep-2024 (Mon)

Lab Timings: 14:00 to 17:00 Hours Pages: 5

Submission time: 16:45 Hrs, 02-Sep-2024

Instructors Dr. V. Vijaya Saradhi

Head TAs Adithya K Moorthy & Laxita Agrawal

Department of CSE, IIT Guwahati

Question 1: (100 points)

Inheritance and computing determinant using LU decomposition.

Task 01 Create a base class named `Matrix` with the following specification

1. (3 marks) Private members
 - (a) `rows` of type `int`
 - (b) `columns` of type `int`
 - (c) A pointer to a pointer to an `float` to store elements of matrix
2. Public members
 - (a) (2 marks) A constructor with the following specification
 - Input argument 1 An `int` with default value of 4 representing number of rows.
 - Input argument 2 An `int` with default value of 4 representing number of columns.
 - Functionality Initialize data members `rows` and `columns` and initialize memory for the double pointer data member.
 - (b) (2 marks) A constructor with the following specification
 - Input argument 1 A `const` pointer to pointer to `float`
 - Input argument 2 An `int` representing number of `rows`
 - Input argument 3 An `int` representing number of `columns`
 - Functionality Initialize data members `rows` and `columns` and initialize memory for the double pointer data member. Copy the contents of the input argument 1 into the data matrix
 - (c) (2 marks) A copy constructor with the following specification
 - Input argument 1 A `const Matrix` reference.
 - Functionality This should initialize the memory for the `Matrix` dynamically. Copy the contents of input argument 1 in this constructor.
 - (d) (20 marks) Declare a `const` function named `lu_decomposition` with the
 - Input: `void`
 - Returns: `pair<Matrix, Matrix>`

- **Functionality:** Given the matrix, decompose it into two matrices L and U. Follow the document `lu-decomposition.pdf` for details. To return a **pair of objects** L and U, implement the following return statement `return make_pair(L, U);`

In the main function you should call this function as `auto [L, U] = A.lu_decomposition();`

- (e) (5 marks) Declare a function named **determinant** with the following specifications.
- **Input argument 1:** `const Matrix` object L
 - **Input argument 2:** `const Matrix` object U
 - **Output:** `float`
 - **Functionality:** Compute determinant of the matrix. Follow the document `lu-decomposition.pdf` for details
- (f) (2 marks) An overloaded **>> operator** with the following specification
- **Input argument 1:** `istream` reference
 - **Input argument 2:** `const Matrix` object
 - **Returns:** `istream` reference
 - **Functionality:** Read the values of the matrix from the input file (no need to use file I/O) into the given object
- (g) (2 marks) An overloaded **<< operator** with the following specification
- **Input argument 1:** `ostream` reference
 - **Input argument 2:** `const Matrix` object
 - **Returns:** `ostream` reference
 - **Functionality:** Print to standard terminal the values of the matrix. Limit the precision to two decimal places.
- (h) (2 marks) A destructor with the following specifications.
- **Input arguments:** None
 - **Returns:** None
 - **Functionality:** Free the dynamically allocated memory of the data member.
- (i) (2 marks) An overloaded *** operator** with the following specification
- **Input argument 1:** `const Matrix` object
 - **Returns:** `Matrix` object
 - **Functionality:** Performs matrix multiplication and return the resulting `Matrix` object.
- (j) (2 marks) An overloaded **- operator** with the following specification
- **Input argument 1:** `const Matrix` object
 - **Returns:** `Matrix` object
 - **Functionality:** Performs matrix subtraction and return the resulting `Matrix` object.

Task 02 Create a derived class called **Identity_Matrix** with the following specification

1. (5 marks) This class must be **public** inherited from the base class **Matrix**
2. (2 marks) A **constructor** with the following specification
 - **Input argument 1** An **int** as default argument with value 4 representing number of rows
 - **Input argument 2** An **int** as default argument with value 4 representing number of columns
 - **Functionality** Initialize data members **rows** and **columns** and initialize memory for the double pointer data member. Initialize the values to identity matrix.
3. (2 marks) A **copy constructor** with the following specification
 - **Input argument 1** A **const Matrix** reference.
 - **Functionality** This should initialize the memory for the matrix dynamically. Copy the contents of input argument 1 in this constructor. Copy must be performed only if the input argument 1 is identity matrix
4. (2 marks) Declare a function named **determinant** with the following specifications.
 - **Input:** **void**
 - **Output:** **void**
 - **Functionality:** Override the base class function with determinant computation of identity matrix.
5. (0 marks) Retain the behavior of **lu_decomposition** as that of the base class
6. (0 marks) Retain the behavior of **determinant** as that of the base class
7. (0 marks) Retain the behavior of overloaded **>> operator** as that of the base class
8. (0 marks) Retain the behavior of overloaded **<< operator** as that of the base class
9. (0 marks) Retain the behavior of overloaded *** operator** as that of the base class
10. (0 marks) Retain the behavior of overloaded **- operator** as that of the base class
11. (0 marks) Retain the behavior of the destructor as that of the base class

Task 03 Create a derived class called **Diagonal_Matrix** with the following specification

1. (5 marks) This class must be **protected** inherited from the base class **Identity**
2. (2 marks) A **constructor** with the following specification
 - **Input argument 1** An **int** as default argument with value 4 representing number of rows
 - **Input argument 2** An **int** as default argument with value 4 representing number of columns

- **Functionality** Initialize data members `rows` and `columns` and initialize memory for the double pointer data member. Initialize the elements of this matrix with all 10.
3. (2 marks) A `copy constructor` with the following specification
 - **Input argument 1** A `const Identity_Matrix` reference.
 - **Functionality** This should initialize the memory for the matrix dynamically. Copy the contents of input argument 1 in this constructor. Copy must be performed only if the input argument 1 is diagonal matrix (irrespective of the value on the diagonal)
 4. (5 marks) Declare a function named `determinant` with the following specifications.
 - **Input:** `void`
 - **Output:** `void`
 - **Functionality:** Override the base class function with `determinant` and call the implementation available in `Matrix` class.
 5. (0 marks) Retain the behavior of `lu_decomposition` as that of the base class
 6. (0 marks) Retain the behavior of `determinant` as that of the base class
 7. (0 marks) Retain the behavior of overloaded `>> operator` as that of the base class
 8. (0 marks) Retain the behavior of overloaded `<< operator` as that of the base class
 9. (0 marks) Retain the behavior of overloaded `* operator` as that of the base class
 10. (0 marks) Retain the behavior of overloaded `- operator` as that of the base class
 11. (0 marks) Retain the behavior of the destructor as that of the base class

Task 04 Create a derived class called `Triangular_Matrix` with the following specification

1. (5 marks) This class must be `private` inherited from the base class `Matrix`
2. (2 marks) A `constructor` with the following specification
 - **Input argument 1** An `int` as default argument with value 4 representing number of rows
 - **Input argument 2** An `int` as default argument with value 4 representing number of columns
 - **Functionality** Initialize data members `rows` and `columns` and initialize memory for the double pointer data member. Initialize the elements of this matrix with all 10.
3. (2 marks) A `copy constructor` with the following specification
 - **Input argument 1** A `const Matrix` reference.
 - **Functionality** This should initialize the memory for the matrix dynamically. Copy the contents of input argument 1 in this constructor. Copy

must be performed only if the input argument 1 is triangular (lower or upper) matrix.

4. (0 marks) Retain the behavior of `lu_decomposition` as that of the base class
5. (0 marks) Retain the behavior of `determinant` as that of the base class
6. (0 marks) Retain the behavior of overloaded `>> operator` as that of the base class
7. (0 marks) Retain the behavior of overloaded `<< operator` as that of the base class
8. (0 marks) Retain the behavior of overloaded `* operator` as that of the base class
9. (0 marks) Retain the behavior of overloaded `- operator` as that of the base class
10. (0 marks) Retain the behavior of the destructor as that of the base class

Task 05 In the `main` function perform the following

1. (1 mark) Declare a `Matrix` object `A`.
2. (2 marks) Read the input from the terminal into `A`.
3. (1 mark) Perform LU decomposition on `A`.
4. (2 marks) Perform `A - (L * U)` and print the same on standard output using overloaded operator `<<`.
5. (1 mark) Compute determinant of `A` and print it.
6. (1 mark) Declare an `Identity_Matrix` object `I`.
7. (1 mark) Perform LU decomposition on `I`.
8. (1 mark) Perform `I - (L * U)` and print the same on standard output using overloaded operator `<<`.
9. (1 mark) Compute determinant of `I` and print it.
10. (1 mark) Declare an `Diagonal_Matrix` object `D`.
11. (1 mark) Perform LU decomposition on `D`.
12. (2 marks) Perform `D - (L * U)` and print the same on standard output using overloaded operator `<<`.
13. (1 mark) Compute determinant of `D` and print it.
14. (1 mark) Declare a `Triangular_Matrix` object `A`.
15. (1 mark) Read the input from the terminal into `A`.
16. (1 mark) Perform LU decomposition on `A`.
17. (2 marks) Perform `I - (L * U)` and print the same on standard output using overloaded operator `<<`.
18. (1 mark) Compute determinant of `A` and print it.